

UCS301 Data Structures

Lab Assignment 1

(Week 1)

1) Develop a Menu driven program to demonstrate the following operations of Arrays

——MENU——

1. CREATE

2. DISPLAY

3. INSERT

4. DELETE

5. LINEAR SEARCH

6. EXIT

Code

```
import numpy as np
arr=np.array([])
while True:
    print("Menu")
    print("1. CREATE")
    print("2. DISPLAY")
    print("3. INSERT")
    print("4. DELETE")
    print("5. LINEAR SEARCH")
    print("6. EXIT")
    choice = input("Enter your choice (1-6): ")
    if choice == '1':
        n = int(input("Enter number of initial elements: "))
        arr = [int(input(f"Enter element {i+1}: ")) for i in range(n)]
        print("Array created successfully!")
    elif choice=="2":
        if not arr:
            print("Array is empty.")
        else:
            print("Current Array:", arr)
    elif choice == '3':
        elem = int(input("Enter element to insert: "))
        pos = int(input(f"Enter position (0 to {len(arr)}): "))
        if 0 <= pos <= len(arr):
            arr.insert(pos, elem)
```

```

        print(f"Inserted {elem} at index {pos}.")
    else:
        print("Invalid position!")
elif choice == '4':
    if not arr:
        print("Array is empty!")
        continue
    pos = int(input(f"Enter index to delete (0 to {len(arr)-1}):
"))
    if 0 <= pos < len(arr):
        removed = arr.pop(pos)
        print(f"Deleted element {removed} from index {pos}.")
    else:
        print("Invalid index!")

elif choice == '5':
    target = int(input("Enter element to search: "))
    found = False
    for i in range(len(arr)):
        if arr[i] == target:
            print(f"Element {target} found at index {i}.")
            found = True
            break
    if not found:
        print(f"Element {target} not found in the array.")

elif choice == '6':
    print("Exiting program. Goodbye!")
    break
else:
    print("Invalid choice! Please select 1-6.")

```

OUTPUT

Menu

- 1. CREATE**
- 2. DISPLAY**
- 3. INSERT**
- 4. DELETE**
- 5. LINEAR SEARCH**
- 6. EXIT**

Enter your choice (1-6): 1

Enter number of initial elements: 2

Enter element 1: 1

Enter element 2: 2

Array created successfully!

Menu

- 1. CREATE**
- 2. DISPLAY**
- 3. INSERT**
- 4. DELETE**
- 5. LINEAR SEARCH**
- 6. EXIT**

Enter your choice (1-6): 2

Current Array: [1, 2]

Menu

- 1. CREATE**
- 2. DISPLAY**
- 3. INSERT**
- 4. DELETE**
- 5. LINEAR SEARCH**
- 6. EXIT**

Enter your choice (1-6): 3

Enter element to insert: 4

Enter position (0 to 2): 2

Inserted 4 at index 2.

Menu

- 1. CREATE**
- 2. DISPLAY**
- 3. INSERT**
- 4. DELETE**
- 5. LINEAR SEARCH**
- 6. EXIT**

Enter your choice (1-6): 2

Current Array: [1, 2, 4]

Menu

- 1. CREATE**

2. DISPLAY

3. INSERT

4. DELETE

5. LINEAR SEARCH

6. EXIT

Enter your choice (1-6): 4

Enter index to delete (0 to 2): 1

Deleted element 2 from index 1.

Menu

1. CREATE

2. DISPLAY

3. INSERT

4. DELETE

5. LINEAR SEARCH

6. EXIT

Enter your choice (1-6): 2

Current Array: [1, 4]

Menu

1. CREATE

2. DISPLAY

3. INSERT

4. DELETE

5. LINEAR SEARCH

6. EXIT

Enter your choice (1-6): 5

Enter element to search: 1

Element 1 found at index 0.

Menu

1. CREATE

2. DISPLAY

3. INSERT

4. DELETE

5. LINEAR SEARCH

6. EXIT

Enter your choice (1-6): 6

Exiting program. Goodbye!

2) Design the logic to remove the duplicate elements from an Array and after the deletion the array should contain the unique elements.

e.g., for A = {1,2,3,4,5} and K = 2, the left rotation is {3,4,5,1,2} and the right rotation is {4,5,1,2,3}.

Code

```
import numpy as np
n = int(input("Enter number of elements: "))
arr = np.array([0] * n)
for i in range(n):
    arr[i] = int(input(f"Enter element {i+1}: "))
temp = np.array([0] * n)
unique = 0
for i in range(n):
    duplicate = False
    for j in range(unique):
        if arr[i] == temp[j]:
            duplicate = True
            break
    if not duplicate:
        temp[unique] = arr[i]
        unique += 1
result = np.array([0] * unique)
for i in range(unique):
    result[i] = temp[i]
print("Original Array:", arr)
print("Array after removing duplicates:", result)
```

OUTPUT

Enter number of elements: 4

Enter element 1: 1

Enter element 2: 2

Enter element 3: 4

Enter element 4: 4

Original Array: [1 2 4 4]

Array after removing duplicates: [1 2 4]

3) Design the logic of a program to rotate a 1-D array by K positions.

Code

```
import numpy as np

n = int(input("Enter number of elements: "))

inp = []

for i in range(n):
    val = input(f"enter elements {i+1}: ")
    inp.append(val)

arr = np.array(inp)

k = int(input("enter positions to rotate: "))

if n > 0:
    k = k % n

left = []

for i in range(n):
    index = (i + k) % n
    left.append(arr[index])

left1 = np.array(left)

right = []

for i in range(n):
    index = (i - k + n) % n
    right.append(arr[index])

right1 = np.array(right)

print("\nOriginal Array :", arr)
print(f"Left Rotation ({k}) :", left1)
print(f"Right Rotation ({k}):", right1)
```

OUTPUT

```
Enter number of elements: 4
enter elements 1: 1
enter elements 2: 2
enter elements 3: 3
enter elements 4: 4
enter positions to rotate: 2
```

```
Original Array : ['1' '2' '3' '4']
Left Rotation (2) : ['3' '4' '1' '2']
Right Rotation (2): ['3' '4' '1' '2']
```

4) Implement the logic to

a. Reverse the elements of an array

Code

```
import numpy as np

n = int(input("Enter number of elements: "))
arr = np.array([0] * n)

for i in range(n):
    arr[i] = int(input(f"Enter element {i+1}: "))

print("original array:", arr)

left = 0
right = n - 1

while left < right:
    temp = arr[left]
    arr[left] = arr[right]
    arr[right] = temp
    left += 1
    right -= 1

print("Reversed Array:", arr)
```

OUTPUT

Enter number of elements: 6

Enter element 1: 1

Enter element 2: 2

Enter element 3: 3

Enter element 4: 4

Enter element 5: 5

Enter element 6: 6

original array: [1 2 3 4 5 6]

Reversed Array: [6 5 4 3 2 1]

b. Find the matrix multiplication

Code

```
import numpy as np

rows_A = int(input("Enter number of rows for Matrix A: "))
cols_A = int(input("Enter number of columns for Matrix A: "))
rows_B = int(input("Enter number of rows for Matrix B: "))
cols_B = int(input("Enter number of columns for Matrix B: "))
```

```

if cols_A != rows_B:
    print("Matrix multiplication not possible! (Columns of A must equal
Rows of B)")
else:
    print("Enter elements for Matrix A")
    A = np.array([[0] * cols_A for _ in range(rows_A)])
    for i in range(rows_A):
        for j in range(cols_A):
            A[i][j] = int(input(f"A[{i}][{j}]: "))
    print("Enter elements for Matrix B ")
    B = np.array([[0] * cols_B for _ in range(rows_B)])
    for i in range(rows_B):
        for j in range(cols_B):
            B[i][j] = int(input(f"B[{i}][{j}]: "))
    result = np.array([[0] * cols_B for _ in range(rows_A)])
    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                result[i][j] += A[i][k] * B[k][j]

    print("\nMatrix A:")
    print(A)
    print("\nMatrix B:")
    print(B)
    print("\nMatrix Multiplication Result:")
    print(result)

rows_A = int(input("Enter number of rows for Matrix A: "))
cols_A = int(input("Enter number of columns for Matrix A: "))
rows_B = int(input("Enter number of rows for Matrix B: "))
cols_B = int(input("Enter number of columns for Matrix B: "))
if cols_A != rows_B:
    print("Matrix multiplication not possible (Columns of A must equal
Rows of B)")
else:
    print(" Enter elements for Matrix A ")

```

```

A = np.array([[0] * cols_A for _ in range(rows_A)])
for i in range(rows_A):
    for j in range(cols_A):
        A[i][j] = int(input(f"A[{i}][{j}]: "))
print("Enter elements for Matrix B ")
B = np.array([[0] * cols_B for _ in range(rows_B)])
for i in range(rows_B):
    for j in range(cols_B):
        B[i][j] = int(input(f"B[{i}][{j}]: "))
result = np.array([[0] * cols_B for _ in range(rows_A)])
for i in range(rows_A):
    for j in range(cols_B):
        for k in range(cols_A):
            result[i][j] += A[i][k] * B[k][j]
print("Matrix A:")
print(A)
print("Matrix B:")
print(B)
print("Matrix Multiplication Result:")
print(result)

```

OUTPUT

```

Enter number of rows for Matrix A: 2
Enter number of columns for Matrix A: 2
Enter number of rows for Matrix B: 2
Enter number of columns for Matrix B: 2
Enter elements for Matrix A
A[0][0]: 1
A[0][1]: 3
A[1][0]: 1
A[1][1]: 4
Enter elements for Matrix B
B[0][0]: 1
B[0][1]: 4
B[1][0]: 2
B[1][1]: 5
Matrix A:
[[1 3]
 [1 4]]

```

Matrix B:

[[1 4]

[2 5]]

Matrix Multiplication Result:

[[7 19]

[9 24]]

Enter number of rows for Matrix A:

c. Find the Transpose of a Matrix

Code

```
import numpy as np
rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))
print(" Enter Matrix Elements")
matrix = np.array([[0] * cols for _ in range(rows)])
for i in range(rows):
    for j in range(cols):
        matrix[i][j] = int(input(f"Element [{i}][{j}]: "))
transposed = np.array([[0] * rows for _ in range(cols)])
for i in range(rows):
    for j in range(cols):
        transposed[j][i] = matrix[i][j]
print("Original Matrix:")
print(matrix)
print("Transposed Matrix:")
```

OUTPUT

Enter number of rows: 3

Enter number of columns: 4

Enter Matrix Elements

Element [0][0]: 53

Element [0][1]: 2

Element [0][2]: 3

Element [0][3]: 4

Element [1][0]: 7

Element [1][1]: 2

Element [1][2]: 7

Element [1][3]: 1

Element [2][0]: 0

Element [2][1]: 3

Element [2][2]: 8

Element [2][3]: 5

Original Matrix:

[[53 2 3 4]

[7 2 7 1]

[0 3 8 5]]

Transposed Matrix:

[[53 7 0]

[2 2 3]

[3 7 8]

[4 1 5]]

5) Write a program to find sum of every row and every column in a two-dimensional array.

```
import numpy as np

rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))
print(" Enter Matrix Elements")
matrix = np.array([[0] * cols for _ in range(rows)])
for i in range(rows):
    for j in range(cols):
        matrix[i][j] = int(input(f"Element [{i}][{j}]: "))
print("Matrix:")
print(matrix)
print("row Sums")
for i in range(rows):
    row_sum = 0
    for j in range(cols):
        row_sum += matrix[i][j]
    print(f"Row {i + 1} Sum = {row_sum}")
print(" Column Sums")
for j in range(cols):
    col_sum = 0
    for i in range(rows):
        col_sum += matrix[i][j]
    print(f"Column{j+1} sum={col_sum}")
```

OUTPUT

Enter number of rows: 2

Enter number of columns: 2

Enter Matrix Elements

Element [0][0]: 1

Element [0][1]: 2

Element [1][0]: 3

Element [1][1]: 4

Matrix:

[[1 2]

[3 4]]

row Sums

Row 1 Sum = 3

Row 2 Sum = 7

Column Sums

Column 1 Sum = 4

Column 2 Sum = 6